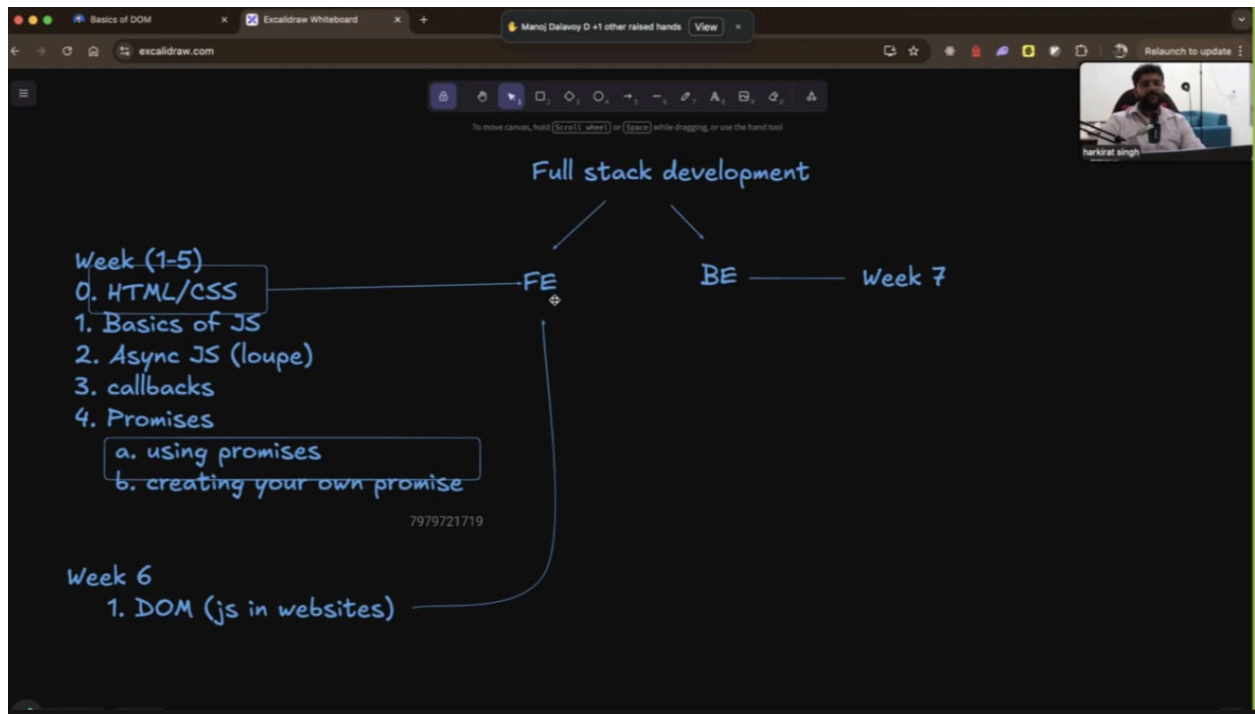


Notes:

<https://petal-estimate-4e9.notion.site/DOM-Manipulation-d43659eddda44472913ecf90e4ebf218>

1. Till Now we studied below in which html,css, dom(js in website) are part of frontend and backend he will start in week 7



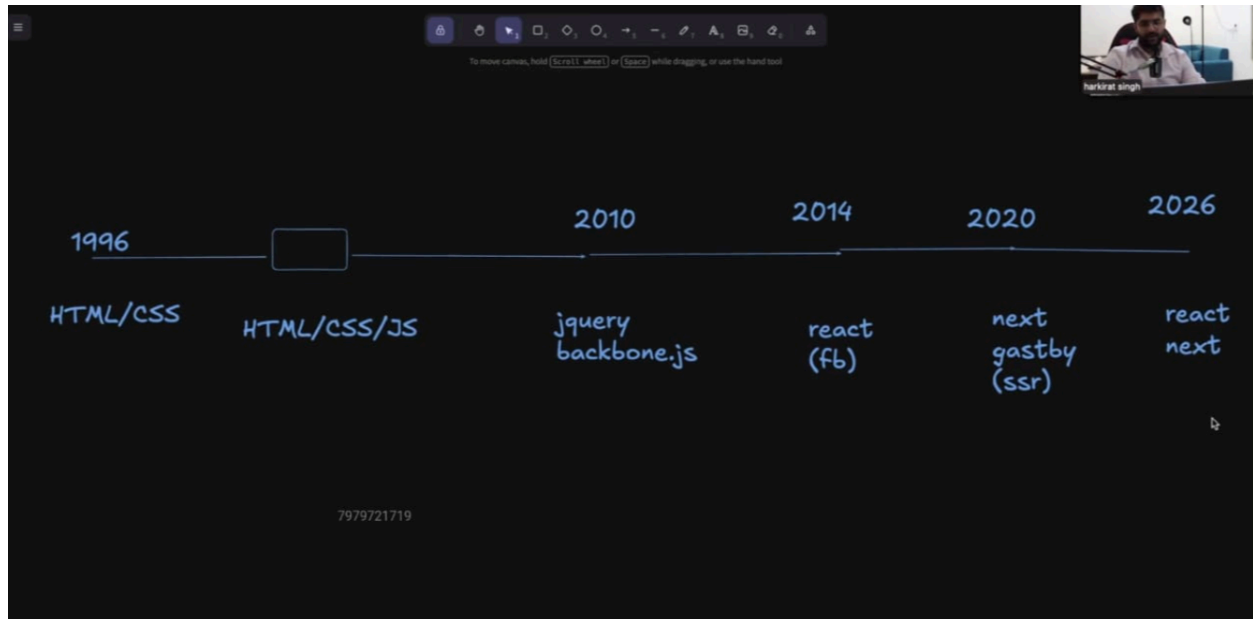
2. In the early days of web development (before 2010), websites were mainly built using **HTML, CSS, and JavaScript**.

Around **2010**, jQuery became very popular. It simplified DOM manipulation, event handling, and AJAX calls, making frontend development much easier and faster.

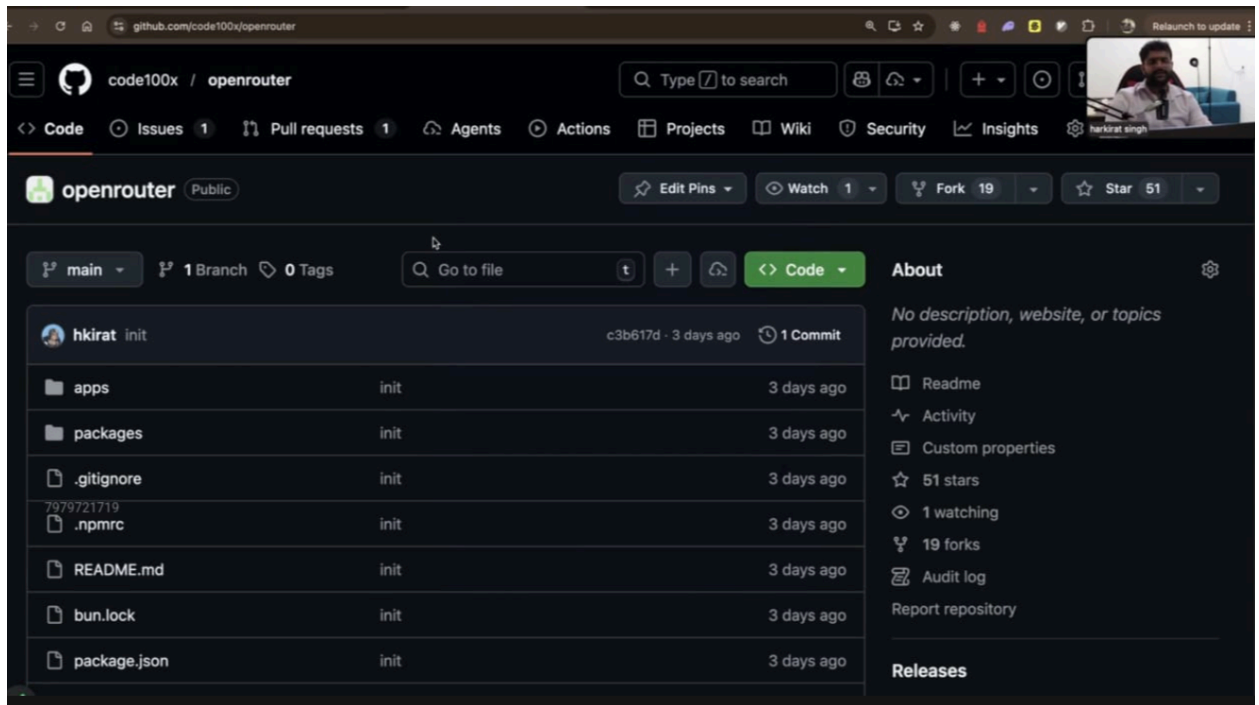
In **2014**, React was introduced and changed the way we build user interfaces. It brought a component-based architecture and made single-page applications (SPAs) more efficient and scalable.

By **2020**, frameworks like Next.js and Gatsby gained strong adoption. They improved performance and SEO through Server-Side Rendering (SSR) and Static Site Generation (SSG).

Now in **2026**, React and Next.js are among the most widely used technologies for building modern web applications, especially for scalable, high-performance frontend and full-stack solutions.



3. There are many bugs in openrouter...harkirat wants us to fix those bugs



4. What is DOM?

DailyCode Basics of DOM (1 / 8) Jump To > < Prev Next > Switch to

What is DOM?

The DOM, or Document Object Model, is a programming interface for web documents. It represents the structure of a web page as a tree of objects.

```
<html>
  <head>
    <title>Simple app</title>
    <meta name="description" />
  </head>
  <body>
    <h1>
      hi there
    </h1>
  </body>
</html>
```

The diagram illustrates the DOM tree structure. The root node is 'html', which branches into 'head' and 'body'. The 'head' node branches into 'title' and 'meta'. The 'body' node branches into 'h1'.

5. Static HTML

Static HTML

As the name suggests, `static HTML` represents HTML that does not change.

For example -

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width">
    <title>replit</title>
    <link href="style.css" rel="stylesheet" type="text/css" />
  </head>

  <body>
    <h1>Todo list</h1>
    <h4>1. Take class</h4>
    <h4>2. Go out to eat</h4>
    <div>
      <input type="text"></input>
      <button>Add Todo</button>
    </div>
    <script src="script.js"></script>
  </body>

</html>
```

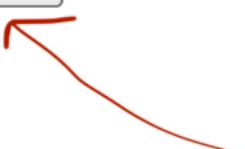
If you click on the `Add Todo` button, nothing happens

Todo list

1. Take class

2. Go out to eat

Add Todo



6. Dynamic HTML

Dynamic HTML

How can you update the elements of the page `dynamically` ?

Assignment

When the user clicks on the `Add todo` button, a new TODO should be added.

Todo list

1. Take class

2. Go out to eat

document object

In the browser, the `document` object is a fundamental part of the Document Object Model (DOM). It represents the web page currently loaded in the browser and provides a way to interact with and `manipulate` its content.

7. JavaScript is a language that can run on both the frontend and the backend.

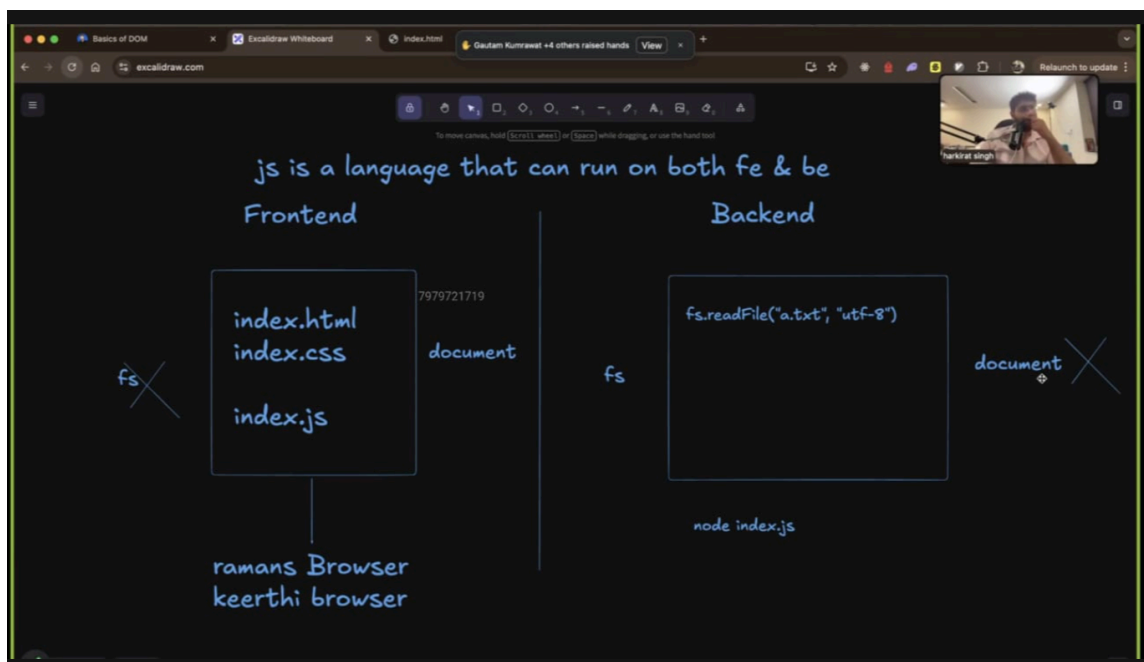
- On the frontend, JavaScript runs inside the browser.
Here, we can use DOM manipulation (like `document.getElementById`, `querySelector`, etc.) to update the UI.
The output is visible directly in the browser.
- On the backend, JavaScript runs using Node.js.
Here, we can use modules like `fs` (File System), for example:

```
fs.readFile("a.txt", "utf-8", callback)
```

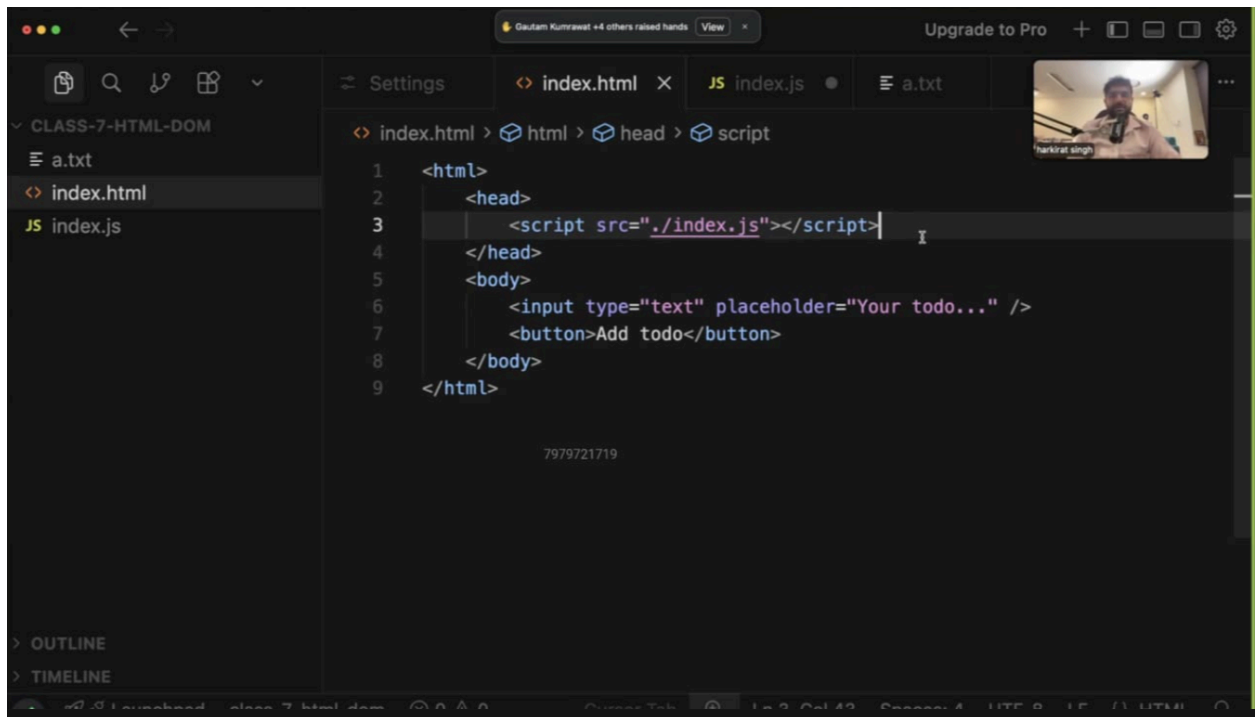
The output of backend code is usually shown in the **VS Code terminal** (or any command-line terminal), not in the browser.

Simple Difference:

- **Frontend JS** → Works with `document` → Output in **Browser**
- **Backend JS (Node.js)** → Works with `fs` → Output in **Terminal**



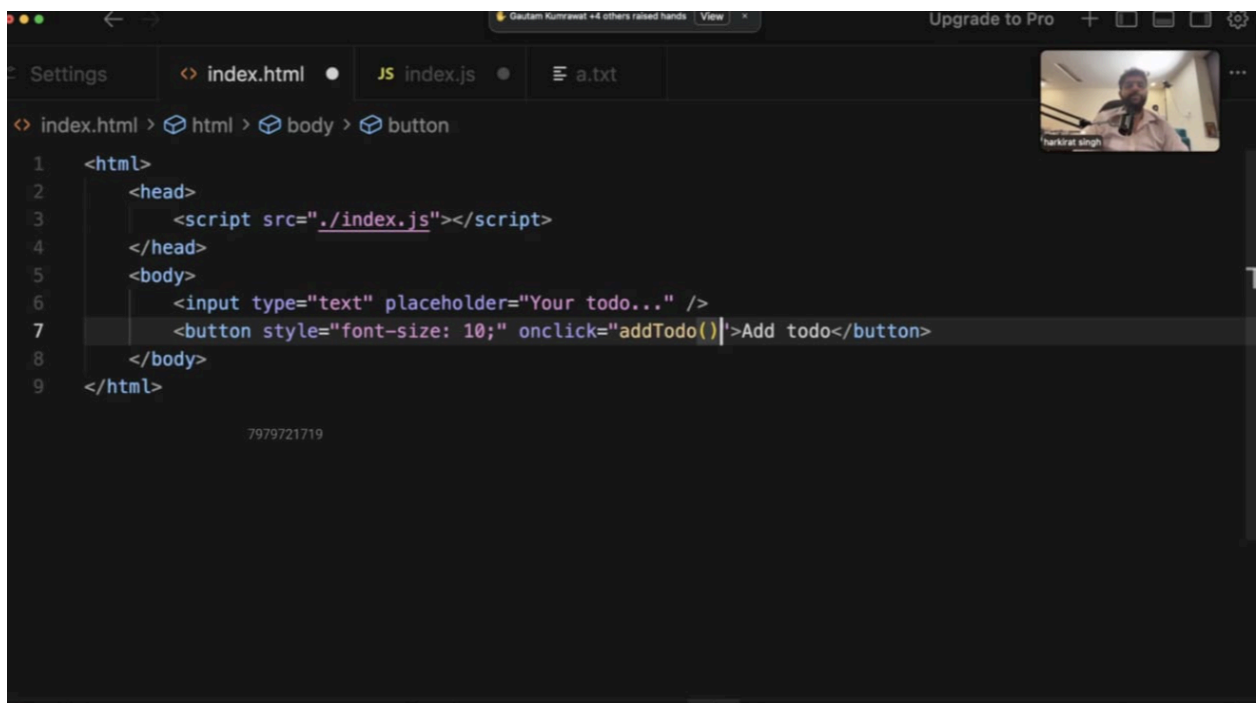
8. Lets make todo application using document



This screenshot shows the VS Code editor with a file explorer on the left containing 'a.txt', 'index.html', and 'index.js'. The breadcrumb navigation at the top of the editor reads 'index.html > html > head > script'. The main editor area displays the following HTML code:

```
1 <html>
2   <head>
3     <script src="./index.js"></script>
4   </head>
5   <body>
6     <input type="text" placeholder="Your todo..." />
7     <button>Add todo</button>
8   </body>
9 </html>
```

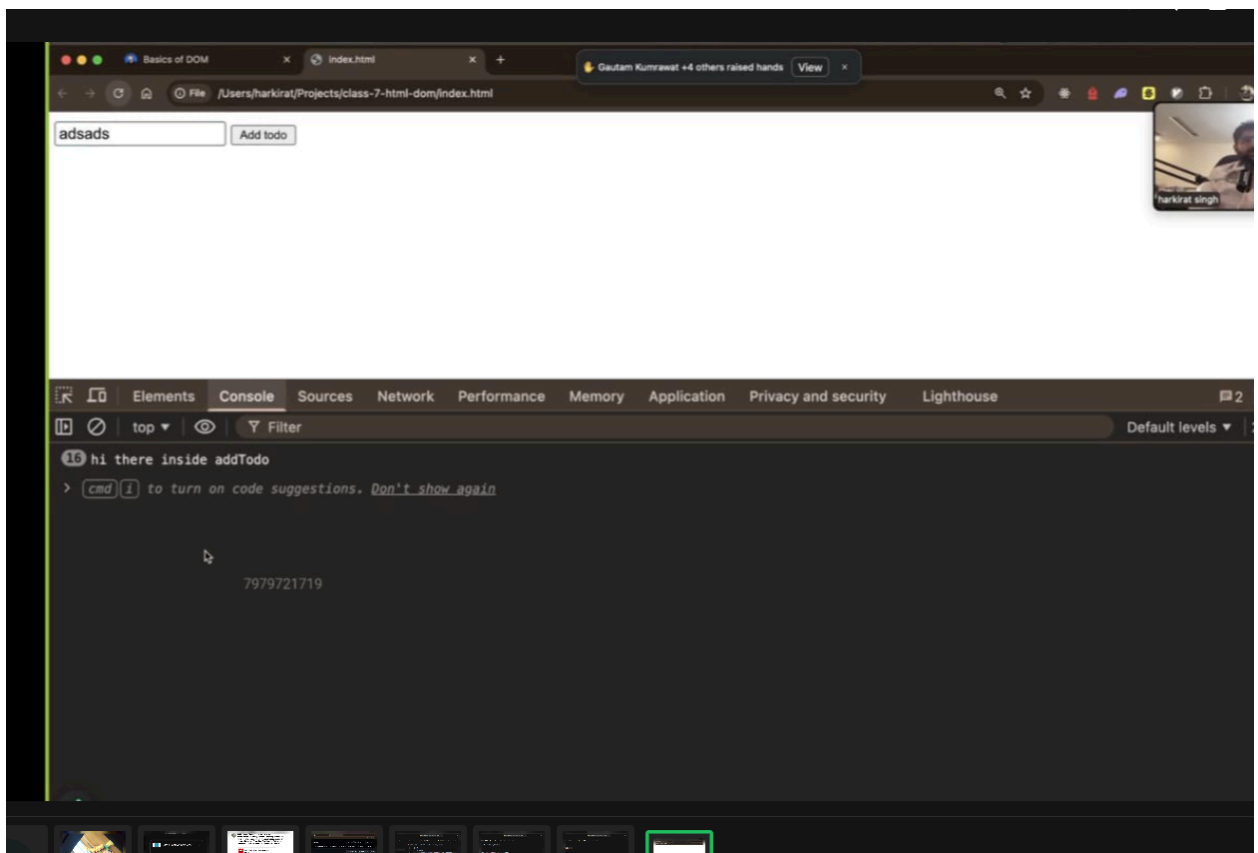
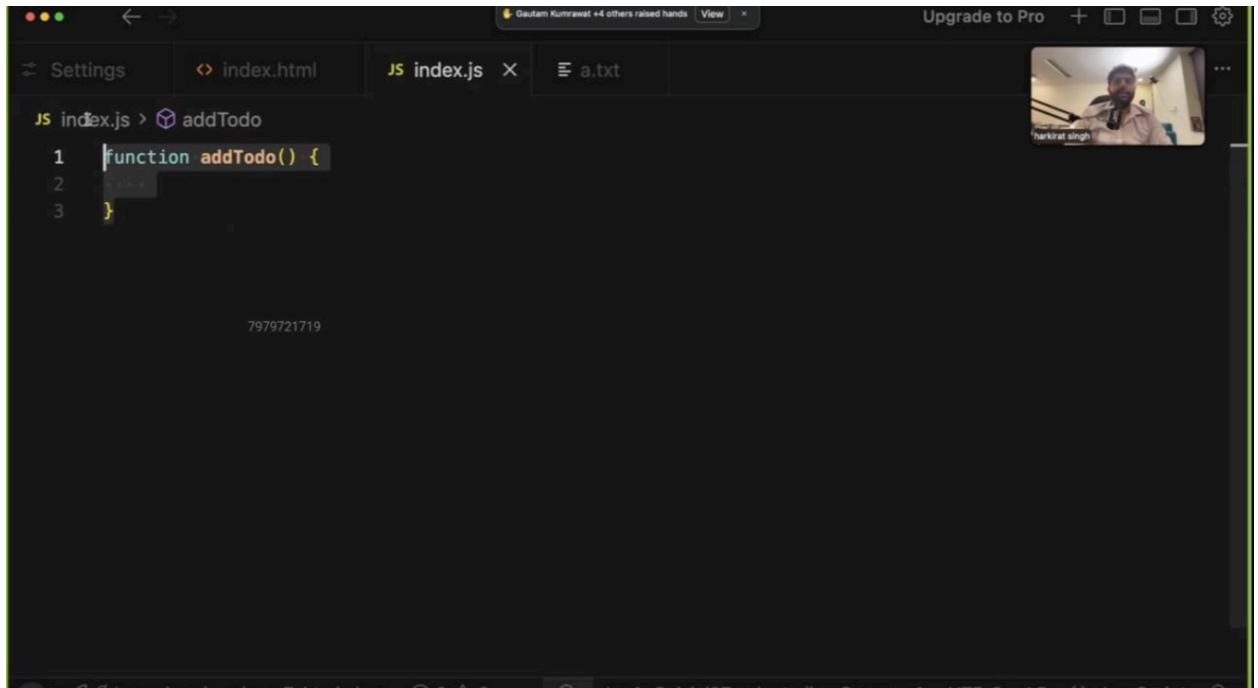
A small video inset in the top right corner shows a person speaking. The status bar at the bottom indicates the file encoding as 'UTF-8' and the language as 'HTML'.



This screenshot shows the VS Code editor with the same file explorer. The breadcrumb navigation at the top of the editor now reads 'index.html > html > body > button'. The main editor area displays the updated HTML code:

```
1 <html>
2   <head>
3     <script src="./index.js"></script>
4   </head>
5   <body>
6     <input type="text" placeholder="Your todo..." />
7     <button style="font-size: 10;" onclick="addTodo()">Add todo</button>
8   </body>
9 </html>
```

The same video inset and status bar are visible as in the previous screenshot.



Settings index.html JS index.js

JS index.js > addTodo

```
1 function addTodo() {
2   // write the code that reads the contents of the input box
3   // creates a new todo on the html dom
4   // clears the input box
5 }
```

7979721719

I

class-7-html-dom

Settings index.html JS index.js a.txt JS index.js

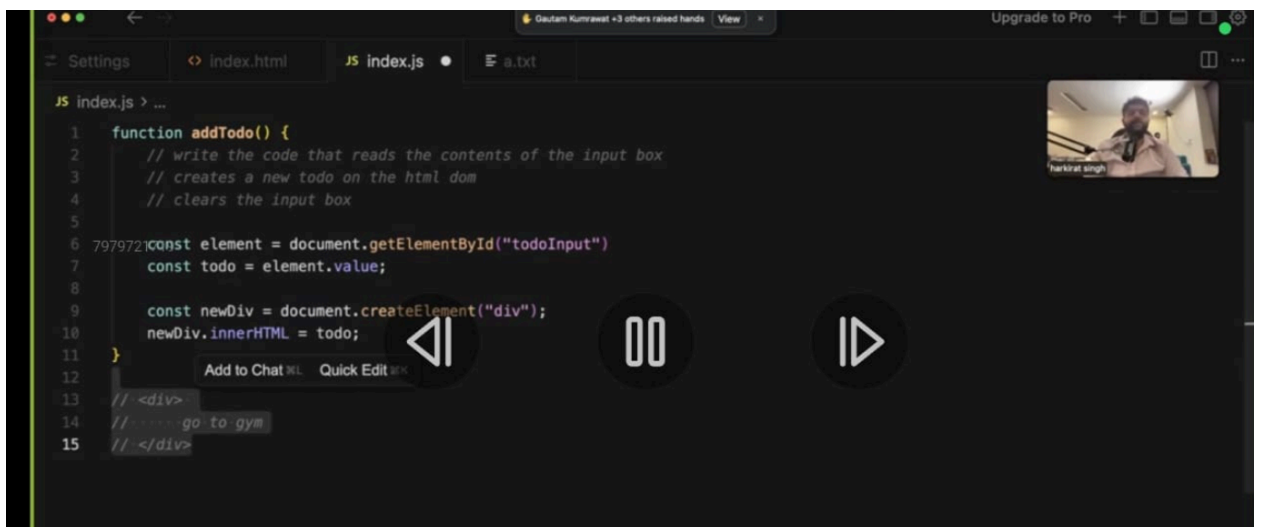
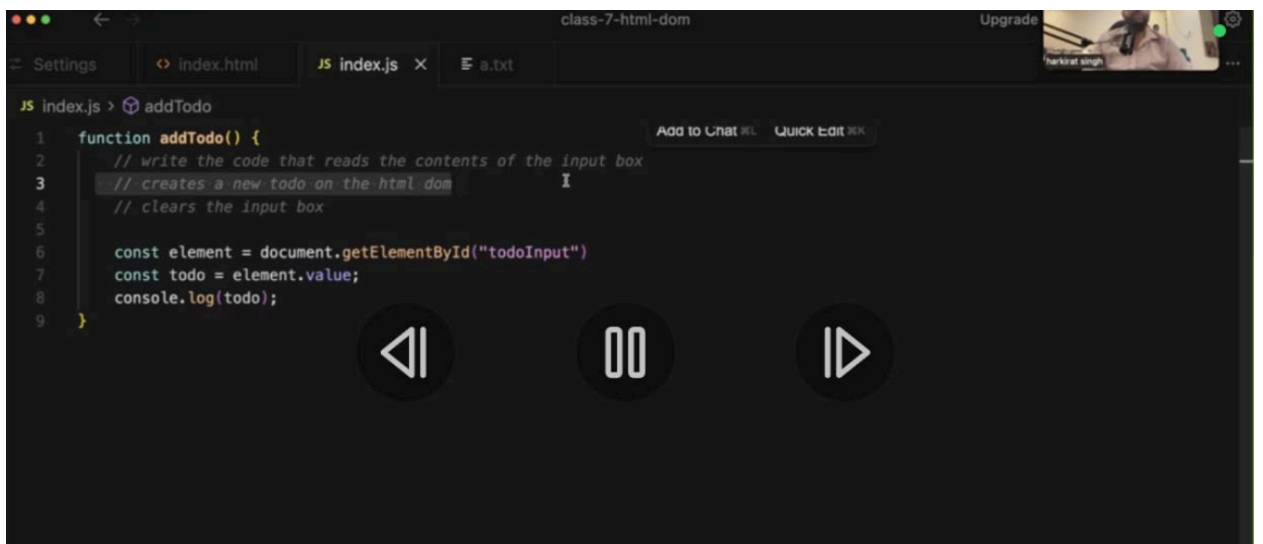
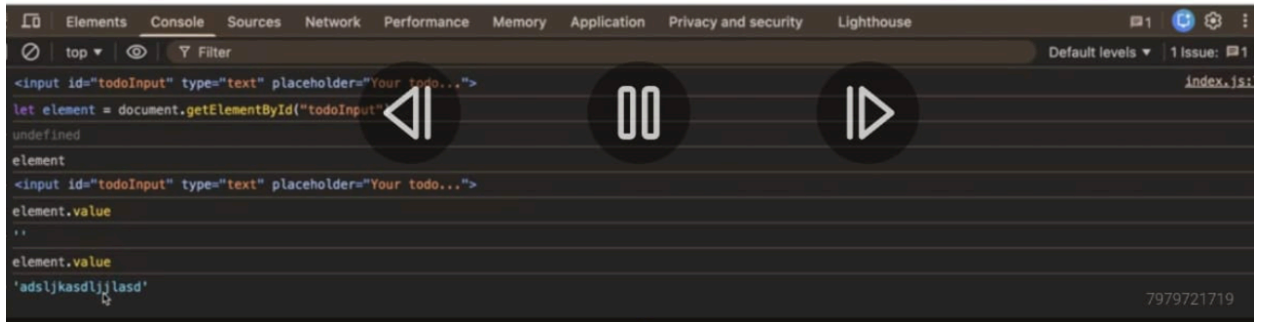
index.html > html > body > input#todoInput

```
1 <html>
2   <head>
3     <script src="./index.js"></script>
4   </head>
5   <body>
6     <input id="todoInput" type="text" placeholder="Your
7     <button style="font-size: 10;" onclick="addTodo()">A
8   </body>
9 </html>
```

JS index.js > addTodo

```
1 function addTodo() {
2   // write the code that reads the contents of the inp
3   // creates a new todo on the html dom
4   // clears the input box
5
6   let element = document.getElementById("todoInput")
7   console.log(element);
8 }
```

Navigation icons: Previous, Play/Pause, Next



Basics of DOM | Index.html | Gautam Kumawat +3 others raised hands | View

File | /Users/harkirat/Projects/class-7-html-dom/index.html

go to gym | Add todo

7979721719

harkirat singh

Elements | Console | Sources | Network | Performance | Memory | Application | Privacy and security | Lighthouse

top | Filter

```
> document.getElementById("todoInput")
< <input id="todoInput" type="text" placeholder="Your todo...">
> let div1 = document.createElement("div")
< undefined
> div1.innerHTML = "go to gym"
< 'go to gym'
> div1
< <div>go to gym</div>
>
```

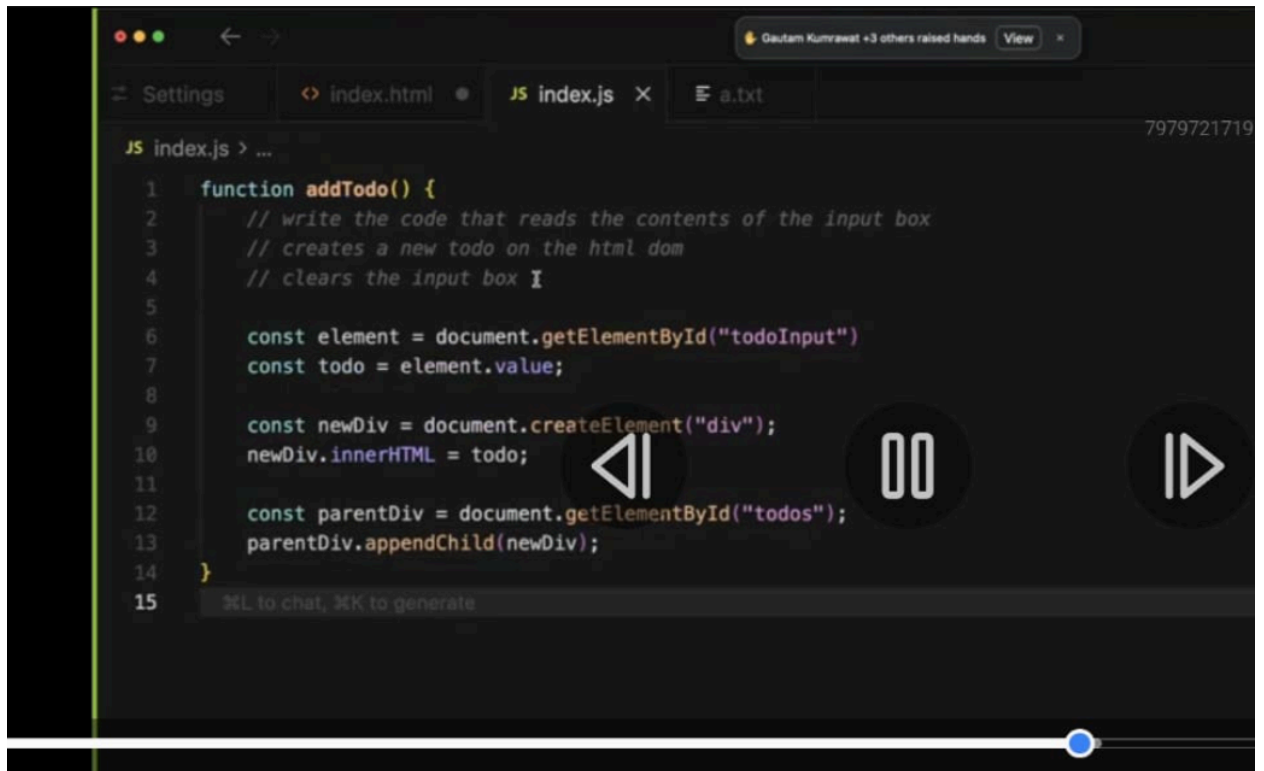
Settings | index.html | JS index.js | a.txt | Upgrade to Pro

index.html > html > body > div#todos

```
1 <html>
2 <head>
3   <script src="/index.js"></script>
4 </head>
5 <body>
6
7   <input id="todoInput" type="text" placeholder="Your todo..." ></input>
8   <button style="font-size: 10;" onclick="addTodo()">Add todo</button>
9
10  <div id="todos">
11    | RL to chat, RLK to get
12  </div>
13
14 </body>
15 </html>
```

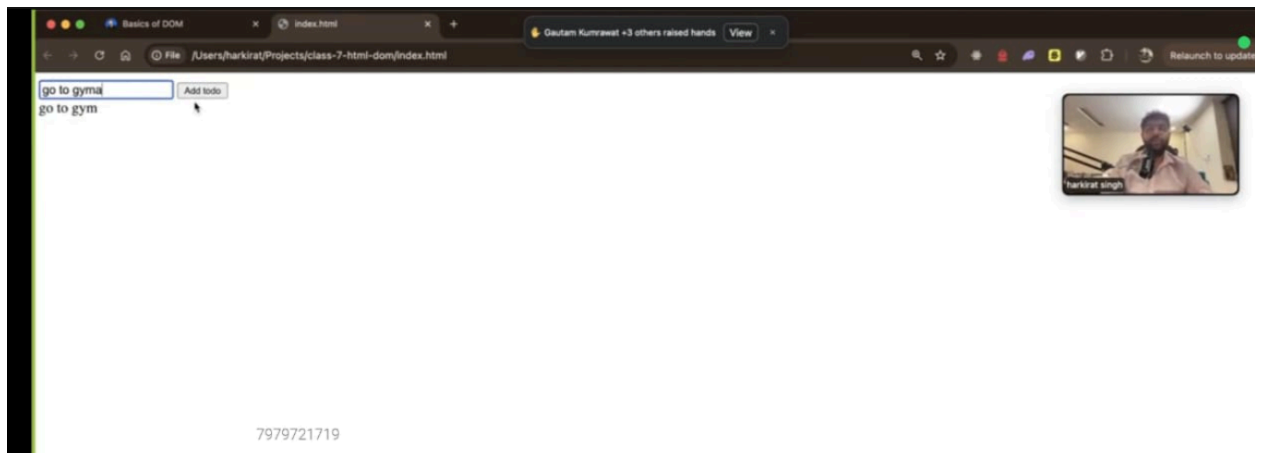
59:19/01:42:03

- Click add button then whatever user input that input get display in div



The screenshot shows the VS Code editor with a file named `index.js` open. The code defines a function `addTodo()` that reads the value from an input box with the ID `todoInput`, creates a new `div` element, sets its `innerHTML` to the input value, and appends it to a parent `div` with the ID `todos`. The code is as follows:

```
1 function addTodo() {  
2   // write the code that reads the contents of the input box  
3   // creates a new todo on the html dom  
4   // clears the input box  
5  
6   const element = document.getElementById("todoInput");  
7   const todo = element.value;  
8  
9   const newDiv = document.createElement("div");  
10  newDiv.innerHTML = todo;  
11  
12  const parentDiv = document.getElementById("todos");  
13  parentDiv.appendChild(newDiv);  
14 }  
15
```



Basics of DOM | Index.html | Escalidraw Wt | Gautam Kurnawat +2 others raised hands | View

escalidraw.com

To move canvas, hold (Shift) while dragging, or use the hand tool

```
⌘L to chat, ⌘K to generate
```

```
const newDiv = document.createElement("div");
newDiv.innerHTML = todo;

const parentDiv = document.getElementById("todos");
parentDiv.appendChild(newDiv);
```

<div>
go to gym
</div>

```
const todoDiv = document.createElement("div")
document.innerHTML = "<span> + todo + </span> <button> Delete Todo </button>"
```

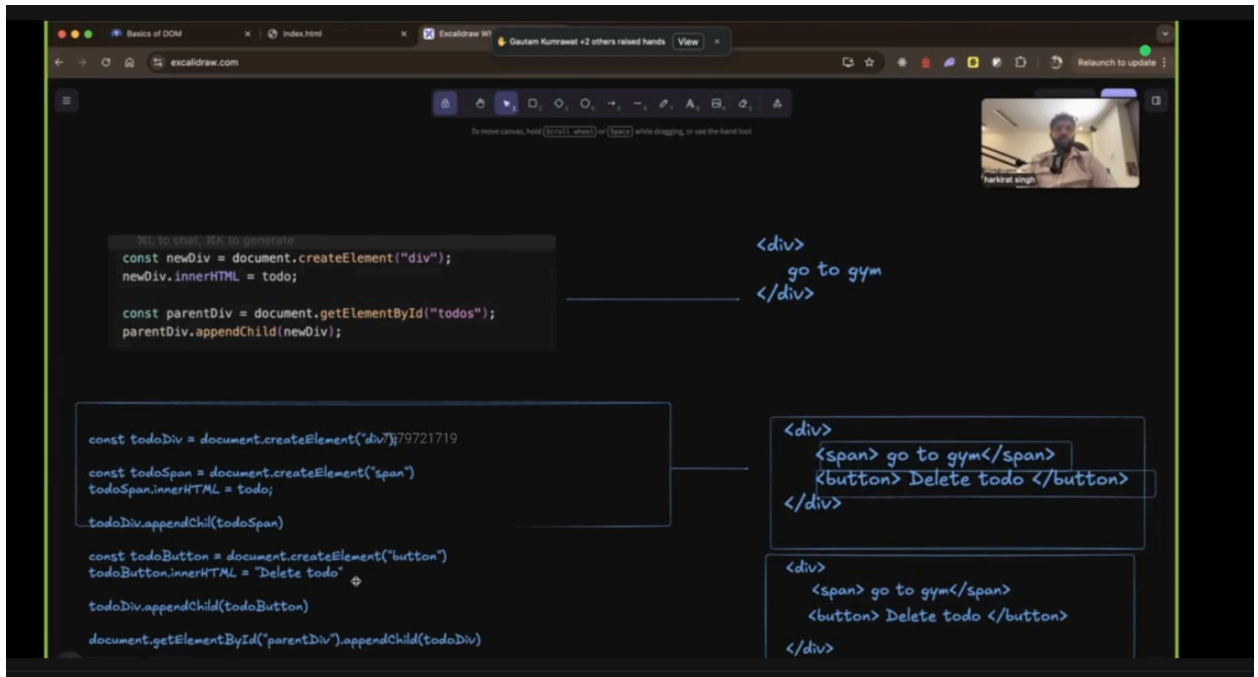
<div>
 go to gym
<button> Delete todo </button>
</div>

Settings | index.html | JS index.js | a.txt

JS index.js > ...

```
1 function addTodo() {
2   // write the code that reads the contents of the input box
3   // creates a new todo on the html dom
4   // step 1 - create a new div element in js (as a variable)
5   // step 2 - insert that div element to a parent div
6   // clears the input box
7
8   const element = document.getElementById("todoInput")
9   const todo = element.value;
10
11   const newDiv = document.createElement("div");
12   newDiv.innerHTML = "<span> + todo + </span> <button> Delete Todo </button>";
13
14   const parentDiv = document.getElementById("todos");
15   parentDiv.appendChild(newDiv);
16 }
17 ⌘L to chat, ⌘K to generate
```

- Create delete button



```
const newDiv = document.createElement("div");
newDiv.innerHTML = todo;

const parentDiv = document.getElementById("todos");
parentDiv.appendChild(newDiv);
```

```
<div>
  go to gym
</div>
```

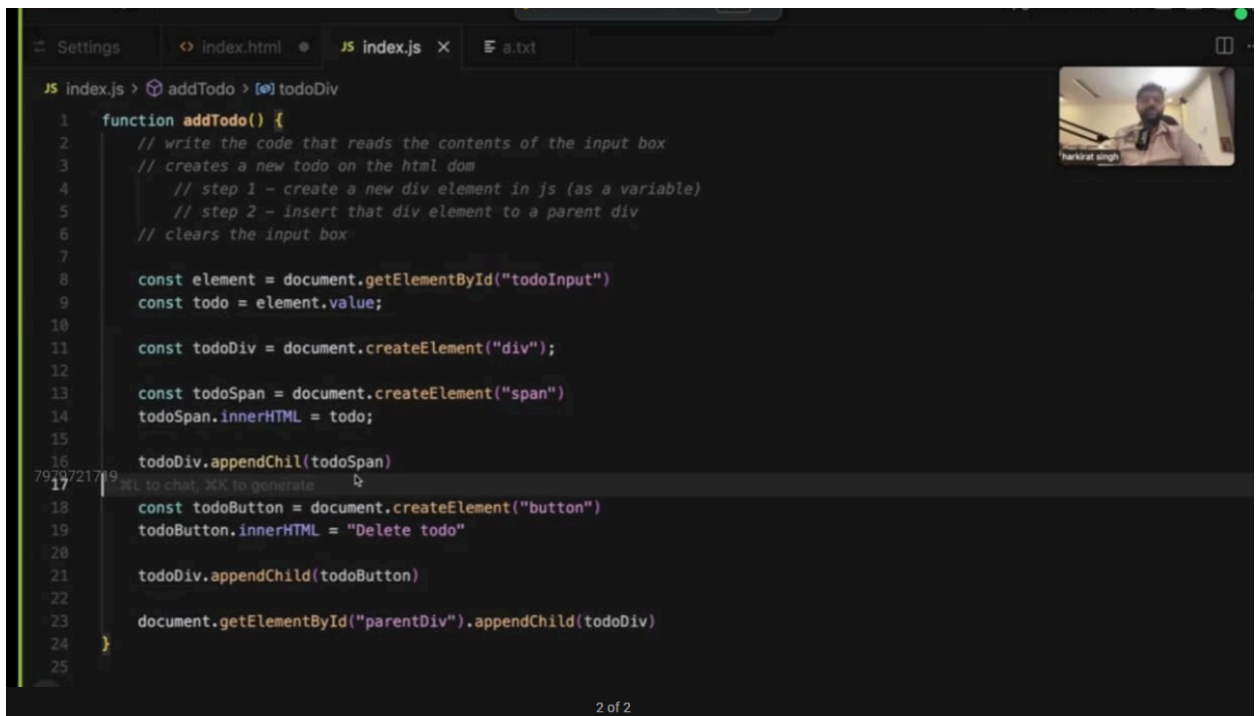
```
const todoDiv = document.createElement("div");
const todoSpan = document.createElement("span");
todoSpan.innerHTML = todo;
todoDiv.appendChild(todoSpan);

const todoButton = document.createElement("button");
todoButton.innerHTML = "Delete todo";
todoDiv.appendChild(todoButton);

document.getElementById("parentDiv").appendChild(todoDiv);
```

```
<div>
  <span> go to gym</span>
  <button> Delete todo </button>
</div>
```

```
<div>
  <span> go to gym</span>
  <button> Delete todo </button>
</div>
```



```
function addTodo() {
  // write the code that reads the contents of the input box
  // creates a new todo on the html dom
  // step 1 - create a new div element in js (as a variable)
  // step 2 - insert that div element to a parent div
  // clears the input box

  const element = document.getElementById("todoInput");
  const todo = element.value;

  const todoDiv = document.createElement("div");

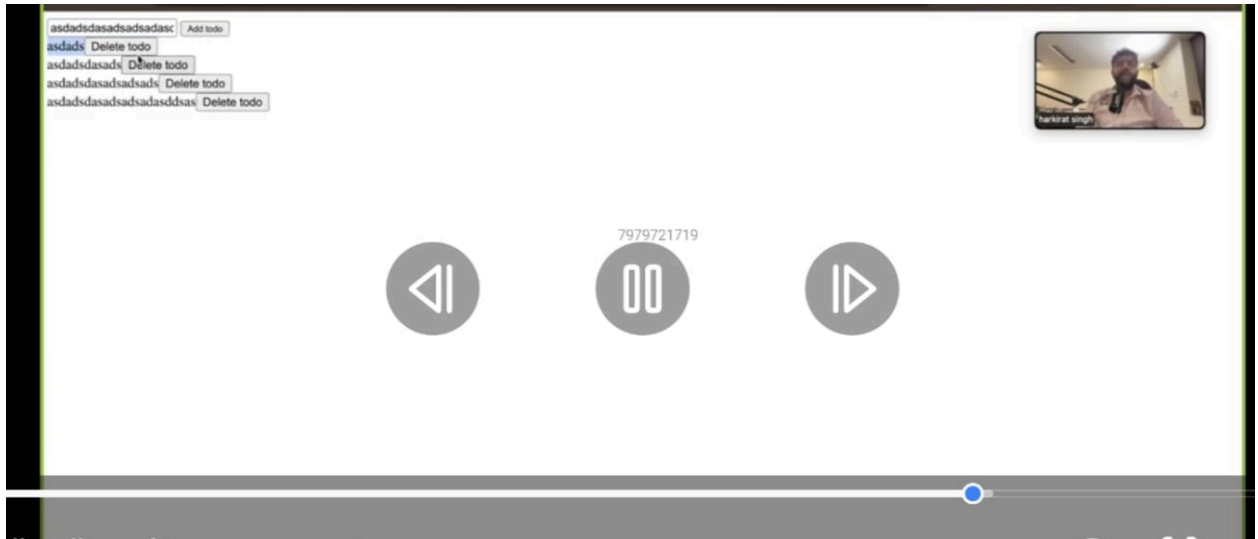
  const todoSpan = document.createElement("span");
  todoSpan.innerHTML = todo;

  todoDiv.appendChild(todoSpan);

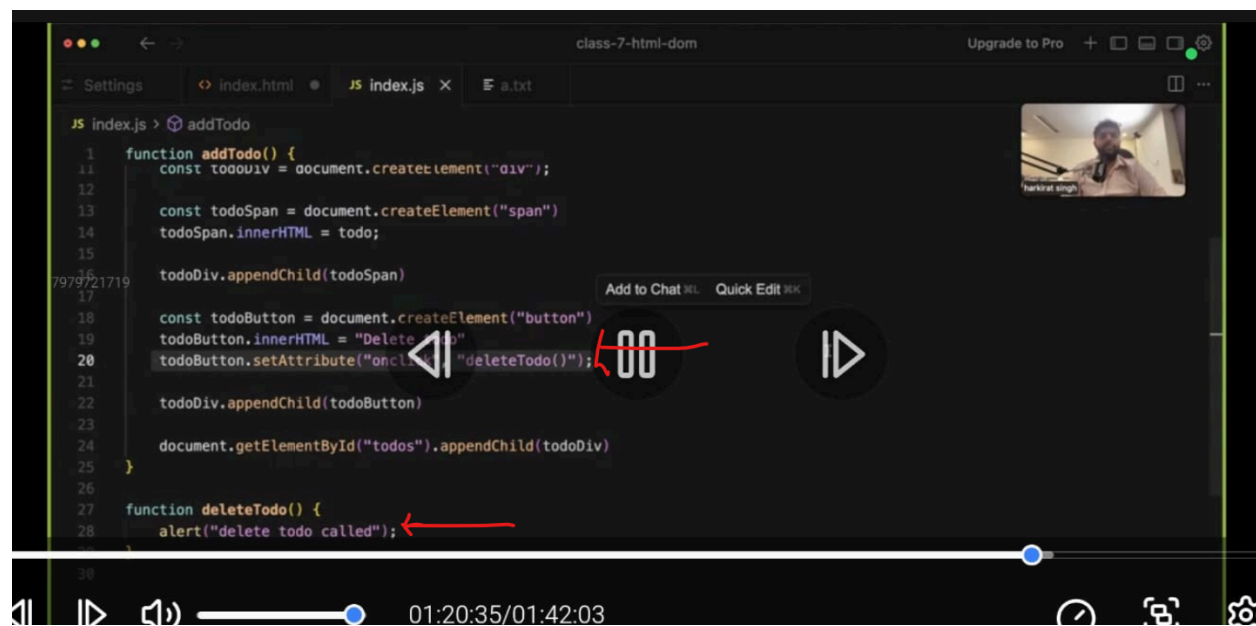
  const todoButton = document.createElement("button");
  todoButton.innerHTML = "Delete todo";

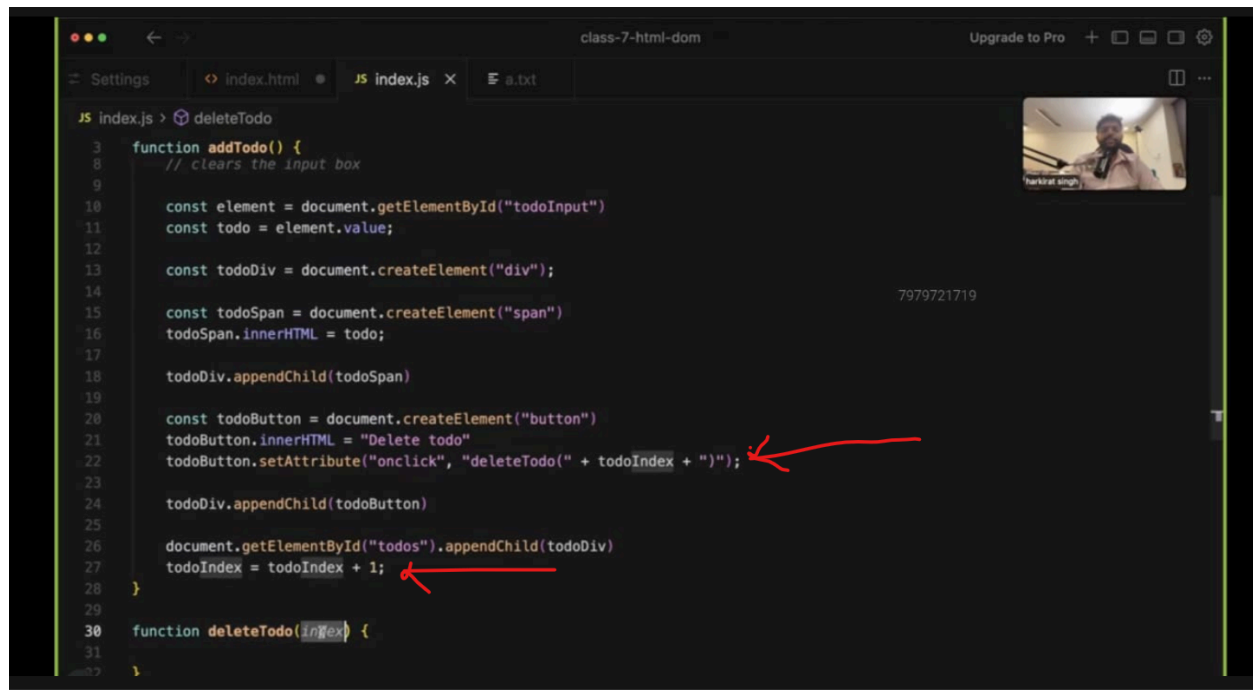
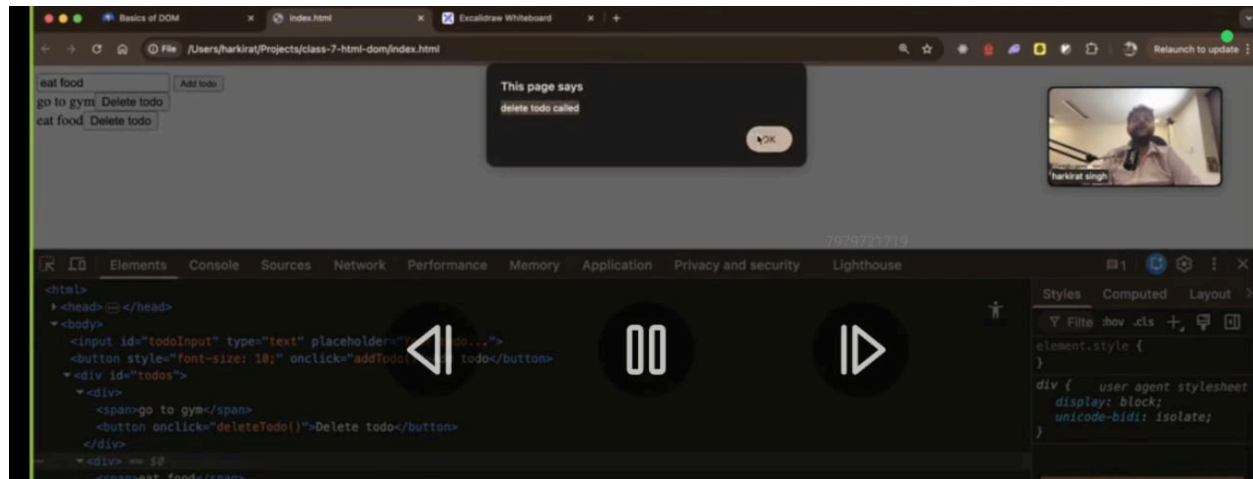
  todoDiv.appendChild(todoButton);

  document.getElementById("parentDiv").appendChild(todoDiv);
}
```



- Lets make delete button functionality i.e onclick delete button it will delete particular todo





class-7-html-dom Upgrade to Pro

Settings index.html JS index.js a.txt

JS index.js > addTodo 7979721719

```
3 function addTodo() {
4     // step 1 - create a new div element in js (as a variable)
5     // step 2 - insert that div element to a parent div
6     // clears the input box
7
8     const element = document.getElementById("todoInput")
9     const todo = element.value;
10
11     const todoDiv = document.createElement("div");
12     todoDiv.setAttribute("id", "todo" + todoIndex);
13
14     const todoSpan = document.createElement("span")
15     todoSpan.innerHTML = todo;
16
17     todoDiv.appendChild(todoSpan)
18
19     const todoButton = document.createElement("button")
20     todoButton.innerHTML = "Delete todo"
21     todoButton.setAttribute("onclick", "deleteTodo(" + todoIndex + ")");
22
23     todoDiv.appendChild(todoButton)
24
25     document.getElementById("todos").appendChild(todoDiv)
26     todoIndex = todoIndex + 1;
27 }
28
29
```

class-7-html-dom Upgrade to Pro

Settings index.html JS index.js a.txt

JS index.js > deleteTodo 7979721719

```
3 function addTodo() {
4     todoSpan.innerHTML = todo;
5     todoDiv.appendChild(todoSpan)
6
7     const todoButton = document.createElement("button")
8     todoButton.innerHTML = "Delete todo"
9     todoButton.setAttribute("onclick", "deleteTodo(" + todoIndex + ")");
10
11     todoDiv.appendChild(todoButton)
12
13     document.getElementById("todos").appendChild(todoDiv)
14     todoIndex = todoIndex + 1;
15 }
16
17 function deleteTodo(index) {
18     // alert("delete todo called with " + index);
19     const divElement = document.getElementById("todo" + index);
20     divElement.parentElement.removeChild(divElement);
21     // alternative way
22     document.getElementById("todos").removeChild(divElement);
23 }
24
25
```